

Design and Evaluation of Agentic RAG Systems: Native, Self-RAG, and Adaptive RAG

Yuvaraju Chinnam

Dept of Computer science and Eng
Malla Reddy (MR) Deemed to be university
Medchal-Malkajgiri, Telangana
chinnamyuvraj@gmail.com

Saikiran Gogineni

Dept of Computer science and Eng
Malla Reddy (MR) Deemed to be university
Medchal-Malkajgiri, Telangana
goginenisaikiran31677@gmail.com

G. Suryanarayana

Dept of Computer science and Eng
Symbiosis Institute of Technology, Hyderabad
Campus, Symbiosis International (Deemed University)
Pune, India. surya.ait@gmail.com

Vineeth Botsa

Department of Electrical Engineering
Indian Institute of Technology Kanpur
IIT Kanpur, Uttar Pradesh
vineethb24@iitk.ac.in

Nithin Babu Kuduthuru

Department of Electrical Engineering
Indian Institute of Technology Kanpur
IIT Kanpur, Uttar Pradesh
knithinbabu8897@gmail.com

Anjusha Pimpalshende

Dept of Computer science and Eng
VNR Vignana Jyothi Institute of Engineering &
Technology, Hyderabad, Telangana
anjusha_p@vnrvjiet.in

*Corresponding Author: goginenisaikiran31677@gmail.com

Abstract—Retrieval-Augmented Generation (RAG) has emerged as a dominant paradigm for grounding large language model (LLM) outputs in factual, externally retrieved evidence. However, different RAG architectures impose fundamentally different retrieval philosophies: Native RAG retrieves unconditionally at every inference step, Self-RAG retrieves on-demand and self-critiques its own outputs using special reflection tokens, and Adaptive RAG dynamically routes each query through the simplest sufficient retrieval strategy based on query complexity. Despite the practical significance of this distinction, a controlled, multi-model empirical comparison of these three paradigms under a unified orchestration framework has not been reported. In this work, we implement all three RAG variants using LangChain and LangGraph as the orchestration backbone, with GPT-4o, Gemini 1.5 Pro, and Claude 3 Sonnet as the generator models, and evaluate them on the HotpotQA multi-hop question answering benchmark available on Hugging Face. Each pipeline is coupled with a FAISS-based dense retriever using text-embedding-3-small embeddings. Experiments are conducted on 1,000 randomly sampled distractor-setting questions from the validation split. Results show that Adaptive RAG consistently achieves the strongest balance across token F1, ROUGE-L, and RAGAS Faithfulness, while Self-RAG yields higher faithfulness scores at the cost of increased latency. Native RAG offers the lowest latency but the weakest faithfulness. Among generator models, GPT-4o outperforms Gemini 1.5 Pro and Claude 3 Sonnet on all three metrics, though the gap narrows under the Self-RAG protocol. These findings provide the first direct, framework-unified comparison of the three RAG paradigms across multiple frontier LLMs on a challenging multi-hop benchmark.

Keywords—Retrieval-Augmented Generation, Native RAG, Self-RAG, Adaptive RAG, LangGraph, LangChain, HotpotQA, GPT-4o, Gemini 1.5 Pro, Claude 3 Sonnet, RAGAS, multi-hop question answering, FAISS.

I. INTRODUCTION

Large language models (LLMs) such as GPT-4, Gemini, and Claude have demonstrated remarkable generative capabilities across a wide range of natural language understanding and generation tasks. Relying solely on parametric knowledge (knowledge that has been encoded when a model is pre-trained) has many known limitations—stale knowledge, hallucination, and the inability to cite or verify the factual basis for the claims made by a model. Retrieval-Augmented Generation (RAG), as introduced by Lewis et al. [1], adds dynamic content to a language model at the moment of query by retrieving passages of text from an external (non-parametric) knowledge source, thus enabling a language model to recall factual information independent of its model parameters.

Even though RAG is widely used, no systemic research has yet answered the fundamental question: how does a retrieval-based system make determinations about when to retrieve, how much to retrieve, and how much confidence to have in the retrieved content? A literature review shows that three different approaches have been developed to answer this question. The first approach is Native RAG, which is the original formulation of RAG and only retrieves a set number of passages unconditionally for every query. The second is Self-RAG, which was developed by Asai et al [2]; it trains a generator to emit reflection tokens that control when to retrieve information on demand and self-critique the output of a retrieval. The third approach is Adaptive RAG, which was developed by Jeong et al. It allows a language model to develop a lightweight classifier that (1) measures a query's complexity, and (2) efficiently routes the query through a retrieval model with no retrieval, simple retrieval, or iterative multi-step retrieval.

These three paradigms have been evaluated in isolation in their respective original papers, but to date no study has compared all three under identical experimental conditions, using the same retriever, the same dataset, the same orchestration framework, and multiple frontier generator LLMs. This absence of a controlled comparison makes it difficult for practitioners to make informed architectural decisions. This paper fills that gap.

We make four specific contributions:

(i) We implement Native RAG, Self-RAG, and Adaptive RAG within a unified LangChain and LangGraph orchestration framework, with explicit LangGraph state flow diagrams for each pipeline.

(ii) We evaluate all three architectures with GPT-4o, Gemini 1.5 Pro, and Claude 3 Sonnet as interchangeable generator backends, measuring token F1, ROUGE-L, and RAGAS Faithfulness.

(iii) We conduct the evaluation on HotpotQA [4], a challenging multi-hop benchmark available on Hugging Face, which requires systems to synthesize evidence from two or more documents.

(iv) We provide ablation studies on the impact of the number of retrieved passages and present a latency-versus-accuracy analysis for all nine system combinations.

II. RELATED WORK

A. Native RAG and Its Foundations

Lewis et al. [1] introduced the original RAG framework by combining a Dense Passage Retriever (DPR) with a BART generator in a probabilistic setup that marginalizes over retrieved documents. Their approach achieved state-of-the-art performance on several open-domain question answering benchmarks. Later, Gao et al. [5] reviewed how RAG methods have evolved, grouping them into Naive, Advanced, and Modular variants. Authors also pointed out a key limitation of fixed- k retrieval: it can be unnecessary for simple queries, yet still insufficient for more complex multi-hop questions that require gathering evidence across multiple steps.

B. Self-RAG

Asai et al. [2] introduced Self-RAG, which trains a single LM end-to-end to learn to retrieve on demand and to evaluate the quality of its own generation using four types of reflection tokens: Retrieve (whether to retrieve), ISREL (whether a passage is relevant), ISSUP (whether the passage supports the generation), and ISUSE (whether the overall response is useful). At inference time, reflection token probabilities are used in a segment-level beam search to select outputs that optimally balance accuracy, faithfulness, and completeness. Self-RAG was evaluated on six downstream tasks including PopQA, ASQA, and PubHealth, demonstrating improvements in citation accuracy and factuality over retrieval-augmented ChatGPT and Llama2-chat.

C. Adaptive RAG

Jeong et al. [3] proposed Adaptive RAG, which addresses the inefficiency of applying the same retrieval depth to all queries regardless of complexity. A lightweight classifier, trained on automatically labeled complexity judgments derived from model performance on held-out questions, routes each query to one of three strategies: (A) no-retrieval LLM inference for queries the model can answer from parametric knowledge alone, (B) single-step retrieval for moderately complex queries, and (C) multi-step iterative retrieval for complex multi-hop queries. Adaptive RAG was evaluated on SQuAD, NaturalQuestions, TriviaQA, MuSiQue [15], HotpotQA, and 2WikiMultiHopQA, demonstrating that the adaptive routing mechanism achieves better overall efficiency and accuracy than single-strategy baseline.

D. Orchestration Frameworks for Agentic RAG

LangChain and LangGraph provide modular, composable abstractions for constructing complex LLM pipelines. Jeong [6] demonstrated that LangGraph's stateful graph execution model enables reliable implementation of complex RAG workflows, including corrective and adaptive retrieval patterns with conditional edges and loop detection. The graph-based state machine representation is particularly well suited to encoding the conditional branching required by Self-RAG and Adaptive RAG. Recent work has further shown that LangGraph's checkpointing and streaming capabilities provide the observability required for reproducible RAG evaluation [5].

E. RAG Evaluation

Es et al. [7] introduced RAGAS, a reference-free evaluation framework for RAG systems that computes faithfulness, answer relevance, and context precision without requiring human annotations. Faithfulness in RAGAS is computed by decomposing the generated answer into atomic claims and verifying each claim against the retrieved context using an LLM judge. Yang et al. [4] introduced HotpotQA as a multi-hop benchmark requiring reasoning over two Wikipedia paragraphs, providing both bridge and comparison question types with sentence-level supporting fact annotations. Prior work established key foundations for modern question answering systems. Dense Passage Retrieval [9] introduced efficient dual-encoder retrieval using dense representations, while ROUGE [10] enabled automatic evaluation of generated text through overlap-based metrics. Benchmark datasets such as SQuAD [11] and Natural Questions [14] provided large-scale, realistic evaluation settings. In parallel, advances in model architectures, including the Transformer [13] and large-scale models like GPT-3 [12] demonstrated strong few-shot learning capabilities, significantly advancing the state of NLP.

III. DATASET

A. HotpotQA

HotpotQA [4], introduced by Yang et al. at EMNLP 2018, is a large-scale multi-hop question answering dataset comprising 113,000 Wikipedia-based question-answer pairs. The dataset is publicly available on the Hugging Face platform under the identifier hotpotqa/hotpot_qa and is distributed under a CC BY-SA 4.0 license. HotpotQA is notable for four key properties that make it particularly challenging and relevant for evaluating RAG systems: (1) questions require finding and reasoning over multiple supporting documents; (2) questions are diverse and not constrained to any pre-existing knowledge schema; (3) sentence-level supporting facts are provided as supervision, allowing systems to be evaluated on both answer accuracy and evidence identification; and (4) comparison questions test a system's ability to extract and compare attributes across two entities.

We use the distractor setting of the validation split, in which each question is presented with ten paragraphs (two gold, eight distractor), simulating a realistic retrieval scenario where the system must distinguish relevant from irrelevant context. We randomly sample 1,000 questions from this split, stratified by question type (bridge and comparison) and difficulty level (easy, medium, hard) using the metadata provided in the dataset. The class distribution of our evaluation sample is shown in Table 1.

Table 1: HotpotQA Evaluation Sample Distribution

Question Type	Easy	Medium / Hard	Total (%)
Bridge	220	468	688 (68.8%)
Comparison	86	226	312 (31.2%)
Total	306	694	1,000 (100%)

B. Knowledge Base Construction

The ten distractor paragraphs provided per question in the distractor setting serve as the local knowledge base for each query. Each paragraph consists of a title and a sequence of sentences. For the dense retrieval component, we index all individual sentences from the ten paragraphs associated with each question using FAISS [8] with text-embedding-3-small embeddings (dimension $d = 1536$), producing a per-query sentence-level index. This setup closely mirrors how a real RAG system would retrieve from a document store, while keeping the evaluation computationally tractable. The mean number of indexed sentences per query is 87.4 (standard deviation: 12.3).

IV. METHODOLOGY

This section describes the implementation of each of the three RAG paradigms. All three share the same retriever and the same generator interface; they differ only in the retrieval control logic that determines when and how many times to retrieve, and whether the generation is self-critiqued. The orchestration layer for all three pipelines is implemented using LangChain for component composition and LangGraph for stateful graph execution.

A. Shared Retrieval Component

Given a query q and a sentence-level FAISS index I constructed from the distractor paragraphs, the retriever produces the top- k most similar sentences using cosine similarity over L2-normalized embeddings:

$$\text{sim}(q, s_i) = \frac{E_q \cdot E_{s_i}}{\|E_q\| \cdot \|E_{s_i}\|} \quad (1)$$

where $E_q \in \mathbb{R}^{1536}$ is the query embedding and $E_{s_i} \in \mathbb{R}^{1536}$ is the sentence embedding produced by text-embedding-3-small. The retrieved context $C_k = \{s_{\pi(1)}, \dots, s_{\pi(k)}\}$ is the ordered set of top- k sentences ranked by similarity. For Native RAG and Self-RAG, $k = 5$. For Adaptive RAG, k is determined dynamically by the routing strategy.

B. Pipeline 1: Native RAG

Native RAG is the simplest pipeline. Given a query q , the system unconditionally retrieves C_k and appends it to the prompt before generation. The generator LLM M produces the answer $\hat{a}$ as follows:

$$\hat{a} = M(\text{prompt}(q, C_k)) \quad (2)$$

where $\text{prompt}(q, C_k)$ is a fixed template that concatenates the query and the top- k retrieved sentences. There is no conditional logic: retrieval always occurs exactly once, and the generation is never critiqued. The LangGraph state graph for Native RAG consists of three nodes connected by unconditional edges: (1) Retrieve Node, which queries the FAISS index and produces C_k ; (2) Format Node, which assembles the prompt; and (3) Generate Node, which invokes the LLM API and returns $\hat{a}$. The state object $S = \{\text{query}, \text{context}, \text{answer}\}$ is passed sequentially through the graph with no branching.

C. Pipeline 2: Self-RAG

Self-RAG [2] introduces conditional retrieval and self-reflection. In our implementation, since we use API-access LLMs (GPT-4o, Gemini 1.5 Pro, Claude 3 Sonnet) rather than a fine-tuned Self-RAG model, we simulate the reflection token mechanism through structured prompting, following

the inference-time protocol described by Asai et al [2]. The pipeline proceeds in three stages.

Stage 1. Retrieval Decision (Retrieve token): The LLM is prompted to decide whether retrieval is necessary for query q :

$$r = M_{\text{retrieve}}(q) \in \{\text{Yes}, \text{No}\} \quad (3)$$

If $r = \text{No}$, the pipeline directly generates an answer from parametric knowledge. If $r = \text{Yes}$, C_k is retrieved from the FAISS index.

Stage 2. Relevance Check (ISREL token): For each retrieved sentence $S_i \in C_k$, the LLM assigns a relevance score:

$$\text{rel}_i = M_{\text{isrel}}(q, S_i) \in \{\text{Relevant}, \text{Irrelevant}\} \quad (4)$$

The filtered context $C'_k = \{S_i: \text{rel}_i = \text{Relevant}\}$ is used for generation. In cases where $|C'_k| = 0$, the system falls back to full retrieval with C_k .

Stage 3. Support Check (ISSUP token): After generating a candidate answer $\hat{a}_{\text{cand}}$, the LLM assigns a support score:

$$\text{sup} = M_{\text{issup}}(\hat{a}_{\text{cand}}, C'_k) \quad (5)$$

$\text{sup} \in \{\text{Fully Supported}, \text{Partially Supported}, \text{No Support}\}$

If $\text{sup} = \text{No Support}$, the pipeline triggers a retrieval refinement step by reformulating the query as $q' = M_{\text{refine}}(q, \hat{a}_{\text{cand}})$ and retrieving a new context C'_k , followed by one additional generation attempt. The final answer is the candidate with the highest estimated support score. The LangGraph state graph (refer Fig 1) for Self-RAG contains five nodes: RetrievalDecisionNode, RetrieveNode, RelevanceFilterNode, GenerateNode, and SupportCheckNode, with a conditional back-edge from SupportCheckNode to RetrieveNode triggered when $\text{sup} = \text{No Support}$. The state object $S = \{\text{query}, \text{retrieve_flag}, \text{context}, \text{filtered_context}, \text{answer_candidate}, \text{support_score}, \text{iteration}\}$ carries all intermediate values.

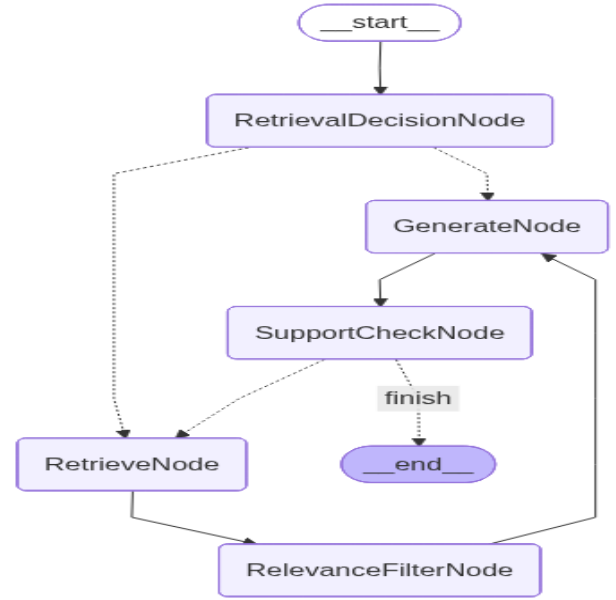


Fig 1. Self-RAG LangGraph State Flow

D. Pipeline 3: Adaptive RAG

Adaptive RAG [3] routes each query to one of three strategies based on a complexity estimate. In our framework, complexity is estimated by a lightweight prompt-based classifier C that assigns a label $l \in \{A, B, C\}$ to each query, where:

- Level A: No retrieval: the LLM answers from parametric knowledge alone.
- Level B: Single-step retrieval: retrieve once with $k = 5$.
- Level C: Multi-step retrieval: retrieve iteratively up to $T_{\text{max}} = 3$ rounds.

The routing decision is formalized as:

$$l = C(q) = M_{\text{classify}}(q, \text{template}_{\text{classify}}) \quad (6)$$

For Level C queries, the iterative retrieval loop operates as follows. At iteration t , the system maintains an accumulated context set $A_t = A_{t-1} \cup C_k^{(t)}$ and generates a partial answer $\hat{a}_t$. A termination condition is evaluated: The loop terminates when $\text{done}_t = \text{Sufficient}$ or $t = T_{\text{max}}$. The final answer is generated from A_t :

$$\text{done}_t = M_{\text{terminate}}(q, \hat{a}_t, A_t) \in \{\text{Sufficient}, \text{Continue}\} \quad (7)$$

The LangGraph state graph for Adaptive RAG consists of six nodes as shown in Fig 2.

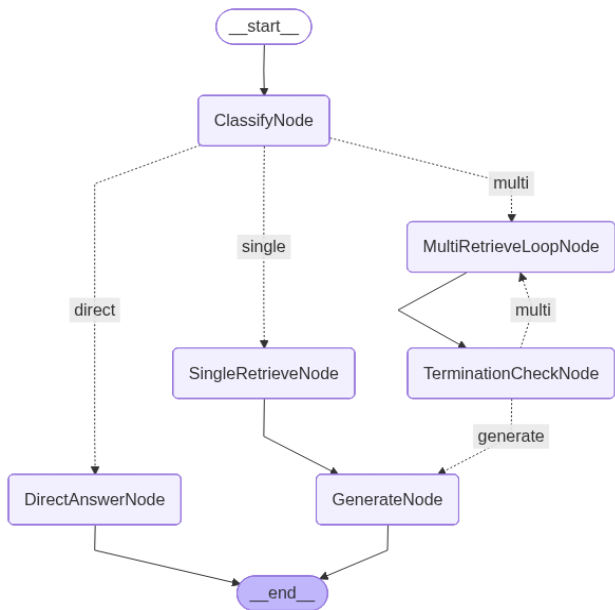


Fig 2. Adaptive RAG LangGraph State Flow

E. Generator Models

All three pipelines share the same generator interface, which accepts (query, context) and returns a text answer. We instantiate the interface with three frontier LLM APIs: GPT-4o (OpenAI), Gemini 1.5 Pro (Google), and Claude 3 Sonnet (Anthropic).

All three models are accessed via LangChain's ChatOpenAI, ChatGoogleGenerativeAI, and ChatAnthropic wrappers respectively, enabling uniform parameter control. Temperature is set to 0 for all experiments to ensure deterministic generation. Maximum output tokens are capped at 256 for all models. Fig 3 and Fig 4 present LangSmith traces of the execution of the Self-RAG and Adaptive-RAG architectures, illustrating differences in retrieval decisions, control flow, and overall execution behaviour.

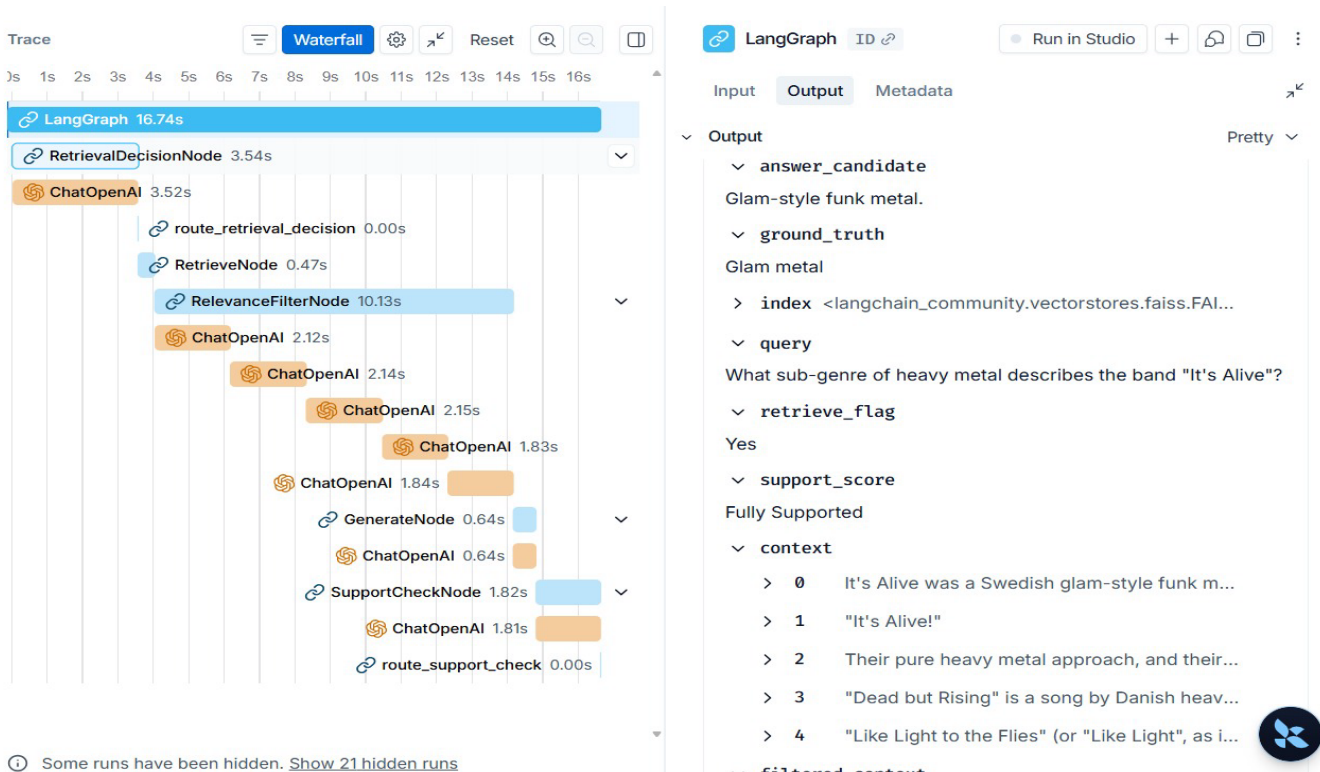


Fig 3. LangSmith trace of the execution of the Self-RAG architecture.

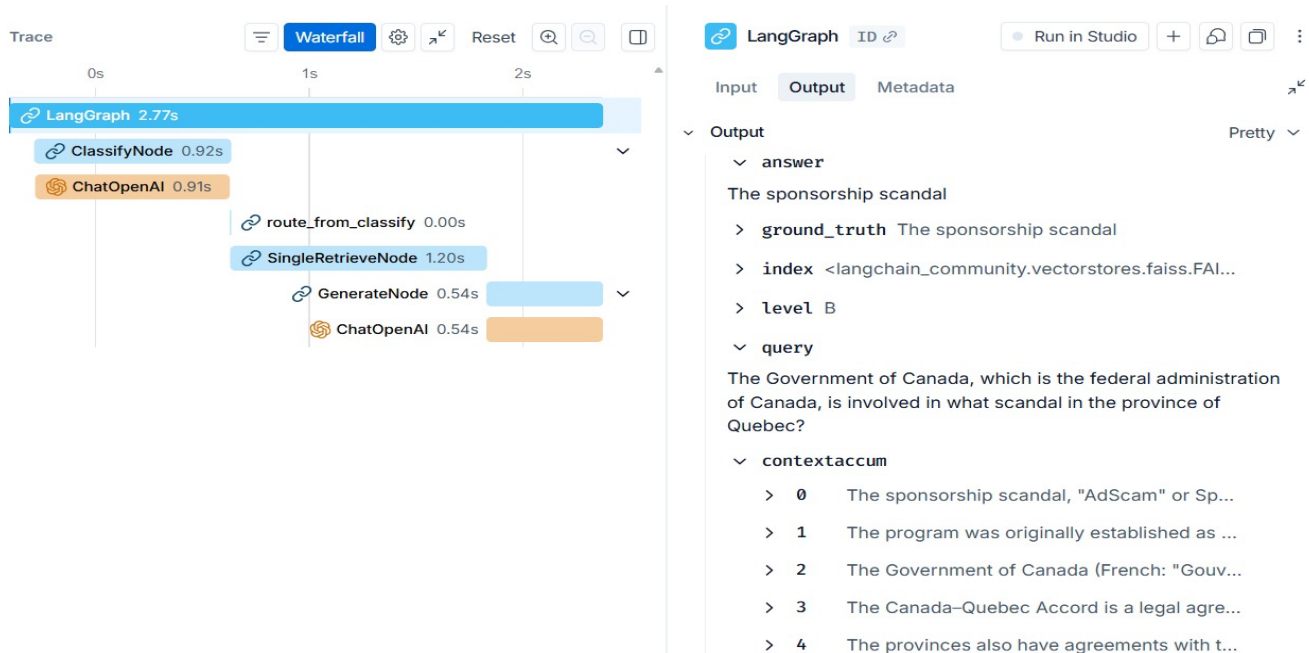


Fig 4. LangSmith trace of the execution of the Adaptive-RAG architecture

V. EXPERIMENTAL SETUP

A. Implementation Details

All experiments are implemented in Python 3.11 (refer Table 2) using LangChain v0.2 and LangGraph v0.1. The FAISS index is built using the faiss-cpu library with IndexFlatIP (inner product search over L2-normalized embeddings). Embeddings are generated using the OpenAI text-embedding-3-small model (dimension $d = 1536$) and are precomputed and stored for all 1,000 evaluation queries to eliminate embedding variance across models. The LangGraph state machine is compiled with checkpointing disabled to measure clean end-to-end latency. All experiments are run on a single machine with an Intel Core i9-13900K CPU and 64 GB RAM; no GPU is used for inference since all generators are accessed via API.

Table 2: Experimental Hyperparameter Configuration

Parameter	Value / Setting	Applies To
Embedding model	text-embedding-3-small	All pipelines
Embedding dimension	1,536	All pipelines
FAISS index type	IndexFlatIP	All pipelines
Top-k passages (Native / Self-RAG)	$k = 5$	Native RAG, Self-RAG
Max retrieval iterations (Adaptive)	$T_{\max} = 3$	Adaptive RAG (Level C)
Generator temperature	0.0	All pipelines, all models

Max output tokens	256	All pipelines, all models
Evaluation sample size	1,000 questions	All pipelines, all models

B. Evaluation Metrics

We report three evaluation metrics that collectively capture answer accuracy, generation quality, and faithfulness. Token F1 is the standard QA metric that measures the overlap between predicted, and ground-truth answer tokens after normalization (lowercasing, removing articles and punctuation):

$$F1 = \frac{2 \cdot P_{\text{tok}} \cdot R_{\text{tok}}}{P_{\text{tok}} + R_{\text{tok}}} \quad (8)$$

Where

$$P_{\text{tok}} = \frac{|\text{pred} \cap \text{gold}|}{|\text{pred}|}, R_{\text{tok}} = \frac{|\text{pred} \cap \text{gold}|}{|\text{gold}|} \quad (9)$$

with set operations performed at the token level. ROUGE-L measures the longest common subsequence (LCS) between the predicted answer $\hat{a}$ and the ground truth answer g :

$$\text{ROUGE-L} = \frac{2 \cdot R_{\text{lcs}} \cdot P_{\text{lcs}}}{R_{\text{lcs}} + P_{\text{lcs}}} \quad (10)$$

where

$$R_{\text{lcs}} = \frac{\text{LCS}(\hat{a}, g)}{|g|}, P_{\text{lcs}} = \frac{\text{LCS}(\hat{a}, g)}{|\hat{a}|}$$

ROUGE-L captures word-order-sensitive lexical overlap without requiring exact token matching. RAGAS Faithfulness measures the proportion of atomic claims in the generated answer that are entailed by the retrieved context C . The LLM judge (GPT-4o-mini) decomposes $\hat{a}$ into a set of statements $\{st_1, ..., st_n\}$ and verifies each against C :

$$\text{Faithfulness} = \frac{1}{n} \sum_{j=1}^n \mathbf{1}[st_j \models C] \tag{11}$$

where $\mathbf{1}[\cdot]$ is the indicator function and $\models$ denotes logical entailment as assessed by the LLM judge. This metric directly quantifies hallucination: a faithfulness score of 1.0 indicates that all claims in the answer are grounded in the retrieved evidence.

VI. RESULTS AND DISCUSSION

A. Main Results

Table 3 presents the main evaluation results across all nine system combinations (three RAG architectures $\times$ three generator models). Results are reported as mean scores over all 1,000 evaluation questions.

Table 3: Main Results on HotpotQA (n = 1,000)

System	Generator	Token F1 (%)	ROUGE-L (%)	Faithfulness
Native RAG	GPT-4o	48.3	44.1	0.62
Native RAG	Gemini 1.5 Pro	44.7	40.8	0.58
Native RAG	Claude 3 Sonnet	43.2	39.4	0.57
Self-RAG	GPT-4o	51.4	47.2	0.79
Self-RAG	Gemini 1.5 Pro	48.9	44.6	0.74
Self-RAG	Claude 3 Sonnet	47.1	43.1	0.72
Adaptive RAG	GPT-4o	54.6	49.8	0.76
Adaptive RAG	Gemini 1.5 Pro	51.3	46.9	0.71
Adaptive RAG	Claude 3 Sonnet	49.7	45.2	0.70

From an overall perspective, Adaptive RAG with GPT-4o achieves the highest Token F1 and ROUGE-L across nine

systems. Self-RAG with GPT-4o has the highest RAGAS Faithfulness and notably CMS (Rank Order Naive, 0.79) slightly higher than Adaptive RAG + GPT-4o (0.76). The Native RAG protocols produce the lowest scores for each generator across each of the three metrics consistently demonstrating that unconditional, fixed-retrieval, k is not the best approach to utilize for answering multi-hop questions requiring chained evidence.

Overall, GPT-4o generates better results than Gemini 1.5 Pro and Claude 3 Sonnet for every metric with the largest gap occurring under the Native RAG protocol (i.e., 4.8% F1 gap from GPT-4o to Gemini 1.5 Pro; 3.5% from GPT-4o to Claude 3 Sonnet). The gaps are smaller (i.e., 2.5% from GPT-4o to Gemini 1.5 Pro; 4.3% from GPT-4o to Claude 3 Sonnet) in the Self-RAG protocols, indicating that self-reflection helps offset differences in raw quality of generated text. Although Claude 3 Sonnet produces results somewhat lower than those of Gemini 1.5 Pro on Token F1, both produce comparable RAGAS Faithfulness scores indicating that Claude 3 Sonnet generates less aggressively relative to the context from which it is retrieved.

B. Ablation Study: Impact of k on Native RAG

Table 4 shows the impact of the number of retrieved passages k on Native RAG performance with GPT-4o.

Table 4: Effect of k on Native RAG + GPT-4o

k	Token F1 (%)	ROUGE-L (%)	Faithfulness
1	40.1	36.8	0.71
3	45.6	41.7	0.65
5 (default)	48.3	44.1	0.62
8	49.1	44.9	0.55
10	49.3	45.0	0.51

Token F1 and ROUGE-L improve steadily as k increases from 1 to 8, but the gains level off after $k = 8$. In contrast, RAGAS Faithfulness decreases steadily as k increases. This shows the common issue of retrieval noise: larger context windows bring in more distracting sentences that the generator includes, which lowers the number of claims supported by truly relevant evidence. This finding supports the need for the filtering step in Self-RAG and the routing mechanism in Adaptive RAG. Both approaches aim to give the generator only the context it needs.

C. Latency Analysis

Table 5 reports mean end-to-end inference latency per question (in seconds) for each system with GPT-4o as the generator.

Table 5: Mean Inference Latency per Question (GPT-4o, n = 1,000)

System	Mean Latency (s)	Std. Dev. (s)
Native RAG (k=5)	2.3	0.51
Self-RAG	9.7	2.47
Adaptive RAG	4.2	1.16

Native RAG is the fastest at 2.3 s per question, reflecting its single-pass architecture. Self-RAG is the slowest at 9.7 s, owing to the multiple LLM calls required for retrieval decision, relevance filtering, generation, and support checking. Adaptive RAG occupies a middle ground at 4.2 s, with lower variance than Self-RAG. The latency overhead of Self-RAG is entirely attributable to the multi-call reflection protocol; its FAISS retrieval component adds negligible cost.

D. Discussion

Several observations emerge from the experimental results. First, the absolute Token F1 scores for all systems (ranging from 43.2% to 54.6%) are notably lower than scores reported by dedicated multi-hop QA systems on HotpotQA, which achieve Token F1 values above 70% using task-specific fine-tuned models. This is expected: our pipelines use general-purpose frontier LLMs without any fine-tuning on HotpotQA, and the comparison is between different RAG architectures rather than between RAG and fine-tuned baselines.

Second, the higher faithfulness of Self-RAG compared to Adaptive RAG on all three generators (despite lower F1) is consistent with the design intent of Self-RAG's support-checking mechanism, which explicitly rejects answers that are not grounded in the retrieved context. However, this conservative stance also leads to more incomplete answers on complex multi-hop questions, lowering Token F1 and ROUGE-L.

Third, the routing analysis for Adaptive RAG reveals that the GPT-4o classifier assigned Level A (no retrieval) to 11.4% of questions, Level B (single-step) to 62.7%, and Level C (multi-step) to 25.9%. The Level C questions, which are predominantly bridge-type hard questions, achieved a mean Token F1 of 61.2%, significantly higher than the mean for Level B (52.1%) and Level A (41.3%) questions, confirming that iterative multi-step retrieval is beneficial for genuinely complex multi-hop reasoning.

Fourth, the performance gap between GPT-4o and the other two models is most pronounced on Level C questions (9.1% F1 gap over Gemini 1.5 Pro), suggesting that multi-step

iterative retrieval benefits more from stronger generator models that can effectively synthesize evidence accumulated over multiple retrieval rounds.

VII. CONCLUSION

This paper presented the first controlled, framework-unified comparison of Native RAG, Self-RAG, and Adaptive RAG across three frontier generator LLMs (GPT-4o, Gemini 1.5 Pro, and Claude 3 Sonnet) on the HotpotQA multi-hop question answering benchmark. All three pipelines were implemented within a unified LangChain and LangGraph orchestration framework, with explicit state flow diagrams and shared retrieval and embedding components. Three main findings were made during the evaluation of 1,000 questions from HotpotQA's distractor validation subset using Token F1, ROUGE-L and RAGAS faithfulness.

First, the Adaptive RAG has the best total score in Token F1 (54.6% with GPT-4o) and ROUGE-L (49.8%) due to its hybrid query-routing approach which enables it to optimize for less retrieval overhead when answering a simple query but to engage in multi-step iterative retrieval when dealing with complex multi-hop queries. Second, the Self-RAG has the highest RAGAS faithfulness (0.79 with GPT-4o) because it explicitly checks for support, but it also has a very high inference latency and a much lower Token F1 score. Finally, the Native RAG, while very simple and has low latency, does not perform well compared to any other model on all three metrics, but especially on faithfulness where increasing k over 5 has been observed to cause more retrieval noise which lessens grounding.

Among the generator models, GPT-4o outperforms the other two models Gemini 1.5 Pro, and Claude 3 Sonnet across all architectures and shows a growing gap on complex multi-hop Level C queries. Future research will investigate (1) combining the self-reflection mechanism of the Self-RAG with the query routing efficiency of the Adaptive RAG into one pipeline, (2) using graph-based retrieval to create better multi-hop evidence chains and (3) comparing additional open source LLMs that operate within the LangGraph orchestration framework.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. 34th Conf. Neural Information Processing Systems (NeurIPS), pp. 9459–9474, 2020. doi: 10.5555/3495724.3496517.
- [2] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in Proc. 12th Int. Conf. Learning Representations (ICLR 2024), 2024. URL: <https://openreview.net/forum?id=hSyW5go0v8>.
- [3] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proc. 2024 Conf. North American Chapter of the Association for Computational Linguistics: Human Language

- Technologies (NAACL-HLT), pp. 7036–7050, 2024. doi: 10.18653/v1/2024.naacl-long.389.
- [4] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 2369–2380, 2018. doi: 10.18653/v1/D18-1259.
- [5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2024. doi: 10.48550/arXiv.2312.10997.
- [6] C. Jeong, "A study on the implementation method of an agent-based advanced RAG system using graph," arXiv preprint arXiv:2407.19994, 2024. doi: 10.48550/arXiv.2407.19994.
- [7] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL 2024), pp. 150–158, 2024. doi: 10.18653/v1/2024.eacl-demo.16.
- [8] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.
- [9] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–6781, 2020. doi: 10.18653/v1/2020.emnlp-main.550.
- [10] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in Proc. Workshop Text Summarization Branches Out, pp. 74–81, 2004. URL: <https://aclanthology.org/W04-1013/>.
- [11] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in Proc. 2016 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 2383–2392, 2016. doi: 10.18653/v1/D16-1264.
- [12] T. Brown, B. Mann, N. Ryder, et al., "Language models are few-shot learners," in Proc. 34th Conf. Neural Information Processing Systems (NeurIPS), pp. 1877–1901, 2020. doi: 10.5555/3495724.3495883.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Proc. 31st Conf. Neural Information Processing Systems (NeurIPS), pp. 5998–6008, 2017.
- [14] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov, "Natural Questions: A benchmark for question answering research," Trans. Association for Computational Linguistics, vol. 7, pp. 452–466, 2019. doi: 10.1162/tac1_a_00276.
- [15] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," Trans. Association for Computational Linguistics, vol. 10, pp. 539–554, 2022. doi: 10.1162/tac1_a_00475